

datalab 报告

姓名：王奕萱

学号：2025201745

总分	bitAnd	bitXor	samesign	logtwo	byteSwap	reverse
37/37	1/1	1/1	2/2	4/4	4/4	3/3

logicalShift	leftBitCount	float_i2f	floatScale2	float64_f2i	floatPower2
3/3	4/4	4/4	4/4	3/3	4/4

test 截图：

```
rm -f *.o btest fshow ishow *~ yacctab.py lextab.py result.txt .autograder_result
gcc -O0 -Wall -std=gnu99 -lm -o btest bits.c btest.c decl.c tests.c
gcc -O0 -Wall -std=gnu99 -o fshow fshow.c
gcc -O0 -Wall -std=gnu99 -o ishow ishow.c
Make success.
bitAnd      1/1:    PASS    detail: Pass using 4 operations including {'~', '|'} and {'int'}.
bitXor      1/1:    PASS    detail: Pass using 7 operations including {'~', '&'} and {'int'}.
samesign    2/2:    PASS    detail: Pass using 9 operations including {'!', '>>', '&&', '^'} and {'int'}.
logtwo      4/4:    PASS    detail: Pass using 18 operations including {'>>', '<<', '>', '|'} and {'int'}.
byteSwap    4/4:    PASS    detail: Pass using 15 operations including {'&', '|', '>>', '~', '<<'} and {'int'}.
reverse     3/3:    PASS    detail: Pass using 5 operations including {'-', '&', '|', '>>', '<<'} and {'while', 'int', 'unsigned'}.
logicalShift 3/3:    PASS    detail: Pass using 6 operations including {'>>', '~', '<<', '&'} and {'int'}.
leftBitCount 4/4:    PASS    detail: Pass using 32 operations including {'!', '+', '>>', '~', '<<'} and {'int'}.
float_i2f    4/4:    PASS    detail: Pass using 19 operations including {'!', '-', '+', '&', '>>', '~', '<', '<<', '>', '=='} and {'while', 'int', 'unsigned', 'if'}.
floatScale2  4/4:    PASS    detail: Pass using 11 operations including {'&', '+', '|', '>>', '<<', '=='} and {'unsigned', 'if'}.
float64_f2i  3/3:    PASS    detail: Pass using 18 operations including {'-', '&', '+', '|', '>>', '~', '<', '<<', '>'} and {'int', 'unsigned', 'if'}.
floatPower2  4/4:    PASS    detail: Pass using 9 operations including {'<<', '-', '+', '<'} and {'int', 'unsigned', 'if'}.
Total points: 37
(base) yxxxx@yxxx:~/CSAPP/datalab2026$
```

解题报告

亮点

logtwo 和 reverse 是我跟着 AI 学下来的，学会之后回头看，这两题我觉得最有亮点。logtwo 是相对最难的一道，一开始走的涂满那条路被运算符限制堵死了，换成二分之后又想了一版连大常量都不用的写法，两种我都实现了。reverse 那个分治的换法挺巧的，五轮依次交换相邻的 1、2、4、8、16 位就能把整个 32 位倒过来，比逐位搬漂亮，我也把它和循环版都写了一遍。

bitAnd 和 bitXor 的亮点在我自己理解的那一层。我是把 32 个位当成一个集合来看，& 是交集、| 是并集、~ 是补集，这样德摩根律在集合和位运算两边说的就是同一件事，题目翻译过去就解开了。这个视角后面几道题也一直在用。

samesign、logicalShift、byteSwap 是完全靠自己的思路做出来的，从想法到实现到通过都没有再问别人。samesign 是想到符号位分不开 0 和正数之后，自己补上了 `!` 这一项；logicalShift 的掩码构造是自己推的，包括怎么绕开 `n` 取 0；byteSwap 的“取出、挖空、放回”三步也是自己想。

下面按我做题的顺序讲，最后四道浮点题放在一起。

bitAnd

这道题和下面的 bitXor 是我最先写的。我感觉把位运算和集合联系起来会好理解一点，就是把一个 int 的 32 个位看成一个全集里的 32 个元素，某一位是 1 就表示这个元素在集合里。这样 `&` 就是交集，`|` 是并集，`~` 是补集。

这题给的是 `~` 和 `|`，也就是补集和并集，而要拼出来的是交集。德摩根律正好干这个事， $A \cap B = \sim(\sim A \cup \sim B)$ ，直接翻译回位运算就是 `~(~x | ~y)`。

其实位运算本身也有德摩根律，可以不绕集合这一圈，但我绕一下自己想得更清楚。

bitXor

还是用集合想。异或对应的是对称差，就是两个圆圈并起来那一整块，再把中间相交的那部分去掉。直接翻译过来就是 `x | y` 再减掉 `x & y`。集合的减法 $A - B$ 就是 $A \cap \sim B$ ，翻译成位运算是先取反再与，所以整个写成 `(x | y) & ~(x & y)`。这题不给 `|`，所以 `x | y` 那里再用一次德摩根换成 `~(~x & ~y)`：

```
return ~(~x & ~y) & ~(x & y);
```

我最开始写的不是这个版本。当时是 AI 提示我，可以从“异或取 1 有哪两种情形”入手，一种是这一位 `x` 为 0、`y` 为 1，另一种反过来。这样很自然地会去分类讨论，写出来是 `(~x & y) | (x & ~y)`，再用德摩根把 `|` 消掉。这个结果是对的，但要八个运算符，超了一个。怎么压到七个我自己没想出来，是问了 AI 才明白的，关键是别去分类讨论，改成“并集减交集”整体地看。

samesign

符号这件事第一反应就是看符号位，所以先把 `x` 和 `y` 都右移 31 位。负数右移完是 `-1`，也就是 32 位全 1；0 和正数右移完都是 0。

问题就出在这儿：0 和正数的符号位是一样的，但题目把 0 单独算一类，光看符号位分不出来。所以还要再找一个能把 0 单独挑出来的东西，那就是 `!`，`!x` 只有在 `x` 为 0 的时候才是 1。于是判据拆成两项，两项都成立才算同号：

```
!((x >> 31) ^ (y >> 31))    /* 符号位相同 */
!((!x) ^ (!y))              /* 是不是 0 这件事也相同 */
```

logicalShift

`x` 是 `int`，`x >> n` 是算术右移，`x` 为负数的时候高 `n` 位补的全是 1，而题目要的逻辑右移那里应该是 0。所以思路很直接，先算术右移，再把高 `n` 位的 1 全部清掉。

清掉就是与上一个掩码，这个掩码要长成“高 `n` 位是 0，剩下的 `32 - n` 位是 1”。造这个掩码不难，`1 << 31` 只有最高位为 1，算术右移之后符号位会一路补过去，再左移一位、取反就得到了：

```
return (x >> n) & ~(((1 << 31) >> n) << 1);
```

byteSwap

这题我觉得挺直观的，三步：把要换的两个字节取出来，把它们原来的位置挖空，再交叉着放回去。

取出来的办法是先右移，把要的那个字节挪到最低的那一个字节的位置，然后和 `0xFF` 与一下就取出来了。第 `n` 个字节对应的移位量是 `n` 乘 8，这里写成 `n << 3`。

挖空用的还是 `0xFF`，只不过反着用：把 `0xFF` 左移到对应的位置去，两个位置或起来，再取个反。这样得到的掩码是“要挖的那两个字节的位置是 0，其他地方全是 1”，和 `x` 与一下，那两个字节就被清干净了，别的字节原样保留。

放回去就是把刚才取出来的两个字节各自左移到对方的位置，再或上去。

```
int n8 = n << 3;
int m8 = m << 3;
int valueofn = (x >> n8) & 0xFF;
int valueofm = (x >> m8) & 0xFF;
int mask = ~((0xFF << n8) | (0xFF << m8));
return (x & mask) | (valueofn << m8) | (valueofm << n8);
```

reverse

要把 32 位整个倒过来。题目里写了可以用 `for` 和 `while`，那我就老老实实循环，一位一位地搬。

做法是另起一个 `result`，每次先左移一位腾出最低位，然后把 `v` 的最低位取出来装进去；`v` 那边取完就右移一位，把已经搬走的那位丢掉。这样循环 32 次，`v` 的最低位是第一个被搬走的，会一路被顶到 `result` 的最高位，正好就是倒序。

```

unsigned result = 0;
int count = 32;
while (count) {
    result = (result << 1) | (v & 1);
    v = v >> 1;
    count = count - 1;
}
return result;

```

AI 还给了一个分治的写法，就是不逐位搬，而是第一轮交换相邻的 1 位，第二轮交换相邻的 2 位，第三轮 4 位，第四轮 8 位，第五轮 16 位，五轮下来整个 32 位就倒过来了。思路上挺简洁的，就是写起来要配一串掩码，稍微麻烦一点。我也把它实现了一遍放在注释里。

logtwo

这题要返回一个正整数以 2 为底的对数，向下取整。我一开始卡在读题上，后来 AI 给我梳理清楚了 `floor(log2(v))` 其实就是在问 `v` 的最高位 1 在第几位，这个转化一做题目就好办了。之前做原版 datalab 的时候有道题就是直接问最高位的，思路可以搬过来。

做那道题的时候我最先想的是课件上 bitCount 那一套，先把最高位以下的位全部涂成 1，也就是 `v |= v >> 1; v |= v >> 2; v |= v >> 4;` 这样一路涂下去，再数一共有多少个 1。但这题给的运算符只有 `> < >> << |`，涂满那一步是能做的，数 1 的个数那一步要靠一串 `&` 掩码，而 `&` 不在允许范围里，所以这条路走不通。

能走的是二分。答案最大是 31，只有五个二进制位，所以拿 `0xFFFF`、`0xFF`、`0xF`、`3` 依次去比，一轮定答案的一个二进制位，同时把已经确定的那一半移走，四轮之后 `v` 只剩 1、2、3，再看一位就够了。这题不给 `+`，但五轮记下的 16、8、4、2、1 占的是答案的不同二进制位，彼此不重叠，所以用 `|` 拼起来和加起来是一样的。

另外这里的大常量其实可以不用。判断 `v` 是不是比 `0xFFFF` 大，等价于问 `v >> 16` 是不是大于 0，因为 `v >> 16` 只要非零就说明 `v` 至少有 17 位。照这个思路把四轮比较全换掉，那些常量就都不需要了：

```

int result, move;
move = ((v >> 16) > 0) << 4; v = v >> move; result = move;
move = ((v >> 8) > 0) << 3; v = v >> move; result = result | move;
move = ((v >> 4) > 0) << 2; v = v >> move; result = result | move;
move = ((v >> 2) > 0) << 1; v = v >> move; result = result | move;
return result | (v >> 1);

```

这一版我也实现了一遍，放在注释里。这一题的二分思路和不用大常量的写法都是 AI 提示的，我自己一开始只想到涂满那条路。

leftBitCount

这题要数从最高位开始连续有多少个 1，和 logtwo 一样是分治，我照着 logtwo 的骨架搬过来的。

差别只在判据。logtwo 问的是“比某个数大吗”，这题问的是“高位是不是全 1”，用 `!(~x >> k) : x` 的高位全是 1 的话，`~x` 的高位就全是 0，右移下来结果就是 0。剩下的过程一样，先试高 16 位，成立就记下 16 并把这 16 位移走，接着问 8、4、2、1。

四道浮点题

最后四道我感觉是完全属于一类的，套路都一样：按 IEEE 754 的规矩把数拆成符号位、阶码 exp 和尾数 frac，然后对 exp 分类讨论，exp 全 1 是 Inf 或者 NaN，exp 全 0 是非规格化数，中间是规格化数，每一类按题目要求做对应的运算就行。难的地方基本都在边界那几类上。

float_i2f

符号位直接取 `x` 的最高位，阶码和尾数来自 `|x|`，把最高位的那个 1 左移对齐之后阶码就定了。两个要小心的地方：`x` 取 `INT_MIN` 时求绝对值会溢出，所以放到无符号域里算；float 只有 23 位尾数，多出来的低位要按就近舍入、中间值取偶来处理。

floatScale2

乘 2 就是阶码加一。exp 为 `0xFF` 是 Inf 或 NaN，原样返回；exp 为 0 是非规格化数，尾数左移一位；exp 为 `0xFE` 再翻倍就溢出成 Inf，要单独拿出来；其余的阶码加一。

float64_f2i

double 的 53 位尾数跨了两个 32 位字，所以要按 $E = \text{exp} - 1023$ 分情况拼。E 小于 0 说明绝对值小于 1，截断成 0；E 太大是溢出，返回 `0x80000000`；中间的看 E 有没有超过 20，决定整数部分要不要跨到低字里去取。

floatPower2

看 2.0^x 落在哪一段。太大返回 `+INF`；落在规格化区就是阶码 `x + 127`、尾数全 0；落在非规格化区，由 $2^{-126} \times \text{frac} / 2^{23}$ 等于 2^x 解出 `frac = 1 << (x + 149)`；再小就返回 0。

反馈/收获/感悟/总结

好难好难好难。做之前先把原版 datalab 做了一遍，结果这版换了一大半题。两版断断续续花了一周。

做法是让 AI 先帮我把思路梳理清楚，我再自己手写一遍，这样强迫我把每一步都想明白。说实话完全靠自己是做不出来的，但做完确实学到了：原来基本都不会，现在至少会了，也能自己写出来，对各种位运算的理解深了不少。

想提一句 ddl。这次和小测撞在一起实在太紧，松一点我们应该能把该吃透的东西更有把握地吃透。

另外这份报告是我口述录音转文字、再让 AI 整理的，语言可能有点 AI 味，内容是我自己的。

参考的重要资料

- 伟大的 AI 老师
- 《深入理解计算机系统》